

Chapter 2

Algorithm Analysis

An important part of learning about data structures is understanding their performance. There are at least two reasons:

Design should take performance into account. What might seem like a natural and easy to code design might be unacceptably slow.

Selection of a data structure for a particular task should also take performance into account.

As you will see, data structures often get fast performance for some operations by sacrificing the performance of others. Being aware of these tradeoffs is part of becoming a good software developer.

There are also two main aspects to algorithm performance: time and space. Time performance is simply how long it takes the algorithm to run, whereas space is concerned with how much memory the algorithm requires. We will concentrate mostly on time.

2.1 Big-O Notation

Big-O notation is an example of an **asymptotic** notation that captures the overall behavior of an algorithm or mathematical function for large input values. In order to get a feel for the usefulness of asymptotic notations, it will help to look at an example.

Suppose we have three algorithms, A, B, and C, which take different numbers of steps depending on their input size n :

Algorithm	Number of Steps
A	$10n + 25$
B	$0.1n^2 + n + 28$
C	n^2

If we compute the number of steps each algorithm takes for different n , the results look like this:

Input Size	Algorithm A	Algorithm B	Algorithm C
10	125	48	100
20	225	88	400
40	425	228	1600
80	825	748	6400
160	1625	2748	25600
320	3225	10588	102400
640	6425	41628	409600
1280	12825	165148	1638400
2560	25625	657948	6553600
5120	51225	2626588	26214400
10240	102425	10496028	104857600

It appears that over the long run, algorithms B and C behave similarly, and that algorithm A is faster than both of them.

We can extract these principles from the table:

Focus on large n Algorithm B is fastest for $n < 100$, but that is not a good indicator of its overall performance. It is much slower than A for large n .

Focus on highest power The columns for n and algorithm A grow very similarly, as do the columns for B and C. What these pairs have in common is their highest power of n : for n and A it is n itself; for B and C it is n^2 .

Ignore other detail Compare the growth of algorithms B and C. The coefficients and smaller terms in $0.1n^2 + n + 28$ do not create an important difference with n^2 itself.

To be realistic, the table should have continued for much larger n , into the millions and tens of millions. You can imagine what that would look like.

Big-O Notation

Big-O notation captures this overall behavior of an algorithm or mathematical function for large n . We say that $g(n)$ is $O(f(n))$ if there is a constant c so that for all large n ,

$$g(n) \leq c \cdot f(n)$$

An algorithm is considered $O(f(n))$ if it takes $g(n)$ steps and $g(n)$ is $O(f(n))$. However, rather than using this precise definition, we will apply the principles derived above to focus on highest powers and ignore other detail.

For example,

Algorithm A is $O(n)$ because $10n + 25$ is $O(n)$. The highest power is n and we ignore the coefficient 10 and lower term.

Algorithm B is $O(n^2)$ because $0.1n^2 + n + 28$ is $O(n^2)$. The highest power is n^2 and we ignore the coefficient and both lower terms.

Algorithm C is $O(n^2)$ because n^2 is itself $O(n^2)$.

Notice how $O()$ captures the sense we had of which algorithms had the “same” performance.

Analyzing an algorithm’s running time using $O()$ notation is known as studying its **time complexity**, while **space complexity** describes memory requirements. Most algorithms have one of the time complexities in Table 2.1, ordered from fastest to slowest.

TABLE 2.1: Common Time Complexities

$O(1)$	Constant
$O(\log n)$	Log
$O(n)$	Linear
$O(n \log n)$	
$O(n^2)$	Quadratic
$O(n^3)$	Cubic

What to Count

Depending on the situation, someone analyzing an algorithm might count different things, such as instructions, comparisons, or element swaps. We will generally think in terms of instructions, but one of the benefits of using $O()$ is that in the end, our specific choice probably won’t affect the result.

Best, Worst, and Average Case

Algorithms may perform very differently on different data sets. Thus, we may be interested in any of these:

Best-case performance assumes that the data is arranged in such a way that the algorithm performs optimally.

Worst-case performance assumes that the data is arranged as badly as possible for this particular algorithm.

Average-case performance averages over all possible data sets.

Normally, best-case performance is not very helpful because it is more about luck than anything else.

Because of the inequality in its official definition ($g(n) \leq cf(n)$), big-O notation is ideally suited for worst-case analysis. Saying that an algorithm is $O(n)$ means that its performance is no worse than linear.

Performance Measurement

A useful counterpart to determining time complexity is to measure an algorithm's performance on an actual machine. **Performance measurement** takes an empirical approach to the question of algorithm performance by measuring the time it takes for code to execute on a particular machine.

Execution time can be measured in Java with the `currentTimeMillis()` function given in Table 2.2. Because it is a static method from another class (the `System` class), this method must be called with a “`System.`” prefix (see page 30).

TABLE 2.2: System Timing

static long <code>currentTimeMillis()</code>
Current time in milliseconds.

The idea is to check the time on a clock immediately before and immediately after running the algorithm:

```
long start = System.currentTimeMillis();
# execute algorithm here
long elapsed = System.currentTimeMillis() - start;
```

Be careful to only time the algorithm in question. Given the speed of current computers, it is common to loop over several executions and then calculate the average time.

An advantage of this approach is that it can be a reality check for an abstract analysis. If your analysis is correct, then real performance times should confirm it. However, beware of machine factors that may impact actual performance, especially caches.

An advantage of $O()$ analysis is that it does not depend on the particular compiler, machine, or in some cases even language, used to code the algorithm. For example, **pseudocode**, which describes algorithms in code-like steps, can often be usefully analyzed using big-O notation.

In the next two sections, we practice algorithm analysis (and using Java arrays) with a pair of examples: insertion sort and binary search.

Exercises

1. Classify these mathematical functions according to their $O()$:

- | | |
|-----------------------------|--------------------------|
| (a) $13n$ | (e) $n^2 + n^3$ |
| (b) $n^2 + n + 100$ | (f) $n^2 + n \log n + n$ |
| (c) 1024 | (g) $n^{3/2} + n$ |
| (d) $\frac{n}{100} + 10000$ | |

2. Determine the $O()$ performance of each of these segments of code. Explain your answers.

(a) **for** (**int** i = 0; i < n; i++) {
 count++;
 }

(b) **for** (**int** i = 0; i < 1000; i++) {
 count++;
 }

(c) **for** (**int** i = 0; i < n; i++) {
 for (**int** j = 0; j < n; j++) {
 count++;
 }
 }

(d) **for** (**int** i = 0; i < n; i++) {
 for (**int** j = i; j < n; j++) {
 count++;
 }
 }

(e) **for** (**int** i = 0; i < n; i++) {
 for (**int** j = 0; j < 1000; j++) {
 count++;
 }
 }

3. Explain what it means for an expression to be $O(1)$.
4. Explain what it means to say that an array provides $O(1)$ access to any element in it.
5. Determine the $O()$ performance of the `factorial()` function in Listing 1.1 in terms of its parameter `n`. Explain your answer.

6. Determine the $O()$ best-case, worst-case, and average performance of the `linearSearch()` function in Listing 1.3. Explain your answers.
7. Determine the $O()$ best-case, worst-case, and average performance of the `sum()` function from Exercise 9 in Section 1.3. Explain your answers.
8. Determine the $O()$ best-case, worst-case, and average performance of the `max()` function from Exercise 10 in Section 1.3. Explain your answers.
9. Determine the $O()$ best-case, worst-case, and average performance of the `min()` function from Exercise 11 in Section 1.3. Explain your answers.

2.2 Sorting: Insertion Sort

Sorting algorithms are an important family of computational algorithms. An array of items named `data` with length `n` is called **sorted** if for all $i < n - 1$,

`data[i] <= data[i+1]`

Such an array is called **nondecreasing**; **nonincreasing** is defined similarly.

Insertion Sort

Insertion sort is often used by card players to sort their hands. The idea is to view the data in two sections: the left section is a sorted subset of the data, while the right section contains items that remain to be sorted. At each step of the algorithm, we take the next element from the right section and insert it into the left, in such a way that the left section remains sorted.

Example

Given the array

0	1	2	3	4	5	6	7	8
3	14	7	22	45	12	19	42	6

begin by viewing the first element as sorted (on its own):

0	1	2	3	4	5	6	7	8
3	14	7	22	45	12	19	42	6

Then take the next element in the right half and insert it in its proper, sorted place on the left. In this case, 14 can stay where it is because $3 < 14$.

0	1	2	3	4	5	6	7	8
3	14	7	22	45	12	19	42	6

The next element, 7, needs to be inserted between 3 and 14. This requires saving the 7, copying 14 into slot 2, and then copying 7 into slot 1:

0	1	2	3	4	5	6	7	8
3	7	14	22	45	12	19	42	6

This process continues until there are no elements on the right, in which case the array is sorted. Inserting one element at a time in this way is an example of an **incremental strategy**.

Implementation

Listing 2.1 implements insertion sort for integer arrays in Java. Pay close attention to the inner loop, which shifts elements in the array so that the key can be inserted at its proper location.

Listing 2.1: Insertion Sort

```

1  // Add to class ArrayFunctions
2  public static void insertionSort(int[] data) {
3      for (int i = 1; i < data.length; i++) {
4          int key = data[i];
5          int j = i - 1;
6          while (j >= 0 && key < data[j]) {
7              data[j + 1] = data[j];
8              j--;
9          }
10         data[j + 1] = key;
11     }
12 }
```

Worst-Case Analysis

Insertion sort has a while-loop inside a for-loop. That makes it difficult to analyze because we need to determine how many times the inner loop runs.

The outer loop on line 3 runs $n - 1$ times on an array of length n . The inner

loop on line 6 depends on the data. In the worst case, the array will be in reverse sorted order, and every item will have to move the largest possible distance. In that case, item i will cause the while-loop to run i times. Therefore, the total number of inner loop iterations in the worst case is:

$$1 + 2 + \cdots + (n - 1)$$

The question is: what is this expression in terms of $O()$? Sums like this occur frequently in algorithm analysis, so it is good to know how to handle them.

A heuristic for finding this sum is to pair its elements from the outside in:

$$\begin{array}{c}
 1 + 2 + 3 + \cdots + (n - 3) + (n - 2) + (n - 1) \\
 \underbrace{\hspace{10em}}_{3 + (n - 3) = n} \\
 \underbrace{\hspace{10em}}_{2 + (n - 2) = n} \\
 \underbrace{\hspace{10em}}_{1 + (n - 1) = n}
 \end{array}$$

Each pair sums to n , and there are $\frac{n - 1}{2}$ pairs, so the total is

$$\text{Pair sum} \times \text{Number of pairs} = \frac{n(n - 1)}{2}$$

which is $O(n^2)$.

Generating Random Data

It would be difficult to test insertion sort (and many other methods) without the ability to generate random test data. Listing 2.2 shows how to use one of the Java libraries to generate random values. It creates a generator object, and then asks the generator for each value to put in the array.

Listing 2.2: Random Data

```

1  // Add to class ArrayFunctions:
2
3  import java.util.Random;
4
5  public static void randomFill(int[] data, int max) {
6      Random gen = new Random();
7      for (int i = 0; i < data.length; i++) {
8          data[i] = gen.nextInt(max);
9      }
10 }
```

The generator is created in line 6, and its `nextInt()` method is called in line 8 to fill in the next array value.

Using Java Libraries

To use Java libraries, it is standard practice to **import** the library classes that you need at the top of your file:

```
import library.name;
```

In Listing 2.2, the name of the library being used is `java.util`, and the name being imported on line 3 is the class `Random`. The full name of the class is `java.util.Random`, and in fact, this name can be used at any time without an **import**. As mentioned, though, it is a good idea to import such names at the top of your class definition files.

A few instance methods from the `java.util.Random` class are listed in Table 2.3. Consult the API documentation for further details and a complete list.

TABLE 2.3: `java.util.Random` Methods

double <code>nextDouble()</code>
Double chosen uniformly from doubles in $[0, 1)$.
double <code>nextGaussian()</code>
Double chosen from normal distribution with mean 0.0 and standard deviation 1.0.
int <code>nextInt()</code>
Integer chosen uniformly from all possible int values.
int <code>nextInt(int n)</code>
Integer chosen uniformly from the set $\{0, 1, 2, \dots, n - 1\}$.

Exercises

1. Finish the example of insertion sort begun on page 38.
2. Explain why the worst case for insertion sort is with reverse-sorted data.
3. Show the operation of insertion sort on these arrays:
 - (a) {31, 7, 2, 34, 10, 5, 40, 22}
 - (b) {29, 12, 48, 41, 19, 6, 25, 33}
 - (c) {21, 31, 39, 22, 18, 38, 25, 6}
 - (d) {24, 43, 42, 33, 37, 31, 40, 8}

4. Use Listing 2.1 to:
 - (a) Explain why the for-loop in line 3 starts at 1 instead of 0.
 - (b) Explain why the array access `data[j]` in line 6 is guaranteed to be valid.
 - (c) Explain why the loop in line 6 runs at most `i` times. (Hint: look at what happens to `j`.)
5. Analyze the best-case performance of insertion sort using $O()$ notation. Explain your work, and describe the data that leads to best-case performance.
6. Analyze the average-case performance of insertion sort using $O()$ notation by considering the average number of times the inner while-loop will run. Explain your work.
7. Add `insertionSort()` from Listing 2.1 to the `ArrayFunctions` class. Include code in `main()` to test insertion sort on a short array of integers.
8. Add `randomFill()` from Listing 2.2 to the `ArrayFunctions` class. Include code in `main()` to test insertion sort on an array of 100 random integers.
9. Write an `isSorted(int[] data)` method in the `ArrayFunctions` class that returns true if the array is sorted and false otherwise. Use it in `main()` to test insertion sort.
10. Write an overloaded version of `randomFill(int[] data)` that does not limit the size of the random integers it fills the array with. Test your method in `main()`.
11. Measure the performance of insertion sort on random integers. Begin with an array of size $n = 100$ and double the array size up to 200,000. Use the version of `randomFill()` from Exercise 10. Report your results and observations.
12. Test these methods from the exercises of Section 1.3 on random integer arrays of size 100:
 - (a) `sum()` method from Exercise 9.
 - (b) `max()` method from Exercise 10.
 - (c) `min()` method from Exercise 11.
 - (d) `display()` method from Exercise 12.

13. **Selection sort** is another natural sorting method that begins by finding the smallest element in the array and swapping it with the first. Then the second-smallest element is found and swapped with the second, and so on until the array is sorted.
- (a) Show the operation of selection sort on the arrays in Exercise 3.
 - (b) Implement a `selectionSort()` method in the `ArrayFunctions` class with tests in `main()`.
 - (c) Determine the $O()$ time complexity of selection sort.
 - (d) Measure the performance of selection sort as in Exercise 11.
 - (e) Compare selection sort with insertion sort in terms of how sensitive the performance of each is to the order of the original data.
-

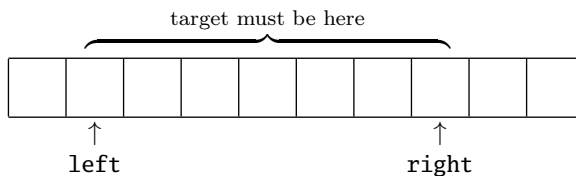
2.3 Searching: Binary Search

We have already seen one search algorithm: linear search in Section 1.3. It works by checking each item, one by one, until it finds the search target. Linear search has $O(n)$ performance, which is reasonable if the list of elements is not too long.

However, if an array is sorted, then a more efficient **divide-and-conquer** strategy can be used. It compares the target with the middle item in the array. Because the array is sorted, if the target is smaller than the middle element, then the entire upper half of the array can be eliminated from consideration. Similarly, the lower half is eliminated if the target was larger than the middle element. Repeating this process leads to the algorithm known as **binary search**.

Implementation

Binary search is delicate, in the sense that it can be difficult to write correctly for all cases.¹ The implementation in Listing 2.3 ensures that if the target is in the array, then it must be trapped between two indices `left` and `right`:



¹See Column 4 from Bentley [2]. An interesting blog post by Bloch [3] points out an error even in that version. The same error is in Listing 2.3 because this code is intended for beginners, but interested readers are encouraged to find and fix it.

The last step may be unexpected. Since `target < data[mid]`, `right` is set to `mid - 1`:

`left: 5` `right: 4`

and this stops the binary search.

Analysis of Binary Search

Listing 2.3 looks difficult to analyze because of the unknown number of repetitions in the loop on line 5. However, we can simplify a worst-case analysis by assuming that the array size n is a power of 2.

If $n = 2^k$, then in the worst case, binary search will divide the array in half approximately k times. To see why, watch the approximate size of the region `data[left...right]`, where the target must be, after each iteration:

	Size of data[left..right]
After 1 iteration	2^{k-1}
After 2 iterations	2^{k-2}
After 3 iterations	2^{k-3}
...	
After k iterations	$2^{k-k} = 2^0 = 1$

This means that in the worst case, the loop runs approximately k times, and so, binary search is $O(k)$ for $n = 2^k$. To obtain an expression in terms of n , we use the fact:

$$\text{If } n = 2^k, \text{ then } k = \log_2 n$$

Thus, binary search is a $O(\log n)$ algorithm.

Exercises

1. Show how binary search works on the array

`{2, 5, 7, 10, 22, 31, 34, 40}`

when searching for each of the elements below. Show the values of `left`, `right`, and `mid` as they change.

- | | |
|--------|--------|
| (a) 31 | (d) 50 |
| (b) 2 | (e) 1 |
| (c) 17 | (f) 34 |

2. Show how binary search works on the array

{10, 13, 24, 36, 37, 41, 44, 66, 86, 100}

when searching for each of the elements below. Show the values of **left**, **right**, and **mid** as they change.

- | | |
|--------|---------|
| (a) 25 | (d) 99 |
| (b) 24 | (e) 100 |
| (c) 86 | (f) 8 |

3. Explain why best-case performance is not helpful for search methods like linear search or binary search.
4. Determine which is faster on unsorted data: linear search, or insertion sort followed by binary search. Assume worst case.
5. Add `binarySearch()` from Listing 2.3 to the `ArrayFunctions` class, and write code in `main()` to call binary search on a short, sorted array of integers. Use an array initializer.
6. Test `binarySearch()` on a sorted array of 100 random integers. Use `insertionSort()` from Section 2.2 to sort the random data.
7. It may seem as though lines 10 and 12 in Listing 2.3 are unnecessary optimizations. Replace those lines of code with the simpler alternatives:

```
right = mid;
```

and

```
left = mid;
```

Describe the result.